# Übungsblatt 6: Multicore-Echtzeit und Shared Memory

**Musterlösung · Echtzeitsysteme (EZS)**

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

## Ziele und Voraussetzungen

Sie begründen eine Mehrkern-Aufteilung, prüfen Speicherlayout und Besitz und unterscheiden Reihenfolge von Cache-Sichtbarkeit. Vorausgesetzt werden Vorlesungen 1–5, insbesondere Job-Antwortzeit, M4-Kernel, lokale Sperren und Pufferübergabe. AMP, Asymmetric Multiprocessing, bedeutet hier zwei getrennte Firmwarekontexte: **Cortex-M7 Bare Metal, Cortex-M4 mit FreeRTOS und CMSIS-RTOS2**. SMP, Symmetric Multiprocessing, würde eine gemeinsame Betriebssystemverwaltung mehrerer Kerne beschreiben.

Teil A enthält **vier klausurnahe Papieraufgaben mit 32 Punkten**, ohne Hardware vollständig lösbar. Planen Sie etwa 15 Minuten für Aufgabe 1, 15 für Aufgabe 2 und 20 für Aufgaben 3–4. Teil B benötigt mit geprüfter Projektbasis etwa 35 Minuten, einschließlich Integration und Auswertung. Installation, erstmaliger Dual-Core-Start und Reparatur eines neuen Linkerlayouts sind Vorbereitung durch die Lehrperson.

Hardware ist das **NUCLEO-H755ZI-Q mit STM32H755ZIT6**. Konfiguration und Generierung erfolgen in **STM32CubeMX**, Build, Flash und Debug in **VS Code mit STM32 VS Code Extension**. Die vollständigen Anwendungsdateien liegen unter `code/lab06/`. Sie ersetzen kein generiertes Firmwareprojekt. Takte, Bootoptionen, Linker-/Startup-Code und tatsächliche MPU-Regionen werden am gestellten Projekt geprüft. Es werden keine bereits erfolgreichen Boardläufe oder Hardwarezeiten vorausgesetzt.

## Teil A: Papieraufgaben

### Aufgabe 1: Softwaretrennung und Zeitbudget · 8 Punkte

1. Die M4-Firmware verwendet einen lokalen FreeRTOS-Kernel, die M7-Firmware eine eigene Hauptschleife. Ist dies das heutige AMP- oder ein SMP-Modell? Darf der M7 eine im gemeinsamen Speicher abgelegte M4-Queue-ID direkt verwenden? Schützt ein M4-Mutex automatisch vor M7-Zugriffen? Begründen Sie jede Aussage. (4 P)
2. Lehrmodell: Eine Reaktion benötigt 4 ms eigene M4-Arbeit, zusätzlich 3 ms Warten auf ein benötigtes M7-Ergebnis und 2 ms Businterferenz. Die Anteile sind ausdrücklich nicht überlappend. Relative Deadline: 8 ms. Berechnen Sie Antwortzeit und Fristabweichung. Warum reicht das Wissen um die eigene M4-Arbeit nicht aus? (2 P)

3. Beide Kerne konfigurieren verschiedene Pins desselben GPIO-Ports durch Lesen, Ändern und Zurückschreiben des gemeinsamen Konfigurationsregisters. Warum genügt die getrennte Pinzuordnung in CubeMX nicht? Nennen Sie eine geeignete Zuständigkeitsregel. (2 P)

**Musterlösung Aufgabe 1**

Die getrennten Firmware- und Kernelkontexte sind AMP. Eine Queue-ID ist ein lokaler Handle der M4-Verwaltung; Wartelisten, Speicherverwaltung und Kernelaufrufe werden nicht durch den Ablageort zu M7-Funktionen. Der M7 benötigt einen ausdrücklichen Kanalvertrag. Ebenso wirkt der M4-Mutex nur für Teilnehmer dieser Schutzregel und schließt den Bare-Metal-M7 nicht automatisch aus.

$R = 4 + 3 + 2 = 9\,\text{ms}$. Die Deadline wird um $9 - 8 = 1\,\text{ms}$ verletzt. Eigene Arbeit ist nur ein Antwortanteil, nicht die gesamte Antwortzeit. Wäre die Busverzögerung bereits in einer anderen Größe enthalten, dürfte sie nicht nochmals addiert werden. Diese Summe gilt nur für das vorgegebene Modell.

Pin Context steuert die Codegenerierung, keine Hardware-Zugriffssperre. Verschiedene Pins können Felder desselben Registers teilen. Zwei Read-Modify-Write-Folgen können eine Änderung gegenseitig überschreiben. Geeignet ist ein Besitzer für das gemeinsame Konfigurationsregister beziehungsweise den Peripherieblock. Gemeinsame Takte initialisiert im Basis-Bootvertrag der M7, bevor der M4 normal weiterläuft.

### Aufgabe 2: Layout, Start und Phasenbesitz · 8 Punkte

Gemeinsamer C-Typ aus der Vorlesung:

```
typedef struct {
    uint32_t magic;
    uint32_t state;
    uint32_t payload;
} ipc_block_t;
```

Jedes Wort hat vier Byte, natürliche Ausrichtung vier Byte. Das Objekt `ipc` wird auf 32 Byte ausgerichtet und seine Sektion auf 32 Byte gepolstert. SRAM4 beginnt bei `0x38000000` und besitzt 64 KiB. Die Kennung wird vor dem M4-Start gesetzt.

1. Geben Sie Feldoffsets, Typgröße, Sektionsreserve und MPU-Regionsgröße an. Warum sind die drei Größen verschieden? (2 P)
2. Die CM7-Map nennt `ipc` bei `0x38000000`, die CM4-Map bei `0x38000020`. Beide verwenden denselben Header. Ist das ein funktionsfähiger gemeinsamer Block? Was muss angepasst und geprüft werden? (2 P)
3. Beginnen Sie mit Payload null und `REQUEST_M4`. Bei jeder Antwort erhöht der M4 um eins. Vor der nächsten Anfrage erhöht der M7 ebenfalls um eins. Notieren Sie drei Antwortwerte, die beiden weiteren Anfragewerte und den endlichen Abschlusszustand. Nennen Sie das Schreibrecht je Phase und die Barrierepositionen. (2 P)
4. Eine Sektion ist `NOLOAD`, das M4-Startup nullt aber trotzdem einen überlappenden Bereich nach dem Aufwachen. Warum ist das falsch? Geben Sie die gültige Init-/Freigabereihenfolge und den Umgang mit einseitigem Core-Reset an. (2 P)

**Musterlösung Aufgabe 2**

Offsets: magic null, state vier, payload acht. Typgröße zwölf Byte, gepolsterte Sektion 32 Byte, MPU-Fenster 64 KiB beziehungsweise 65536 Byte. Die Typgröße beschreibt die Felder, die Sektionsreserve isoliert und richtet das Objekt aus, die MPU beschreibt eine größere Region. Objektalignment ändert nicht automatisch die natürliche Typgröße.

Die Maps bezeichnen verschiedene Bytes. Gleicher Name und Header reichen nicht. Beide tatsächlichen Linkerdateien müssen das einzige Objekt an denselben vereinbarten Beginn legen. Prüfen Sie die ausgewählte Datei im CMake-Linkaufruf, Sektion, Assertions, andere Allokationen und beide resultierenden Maps. Private Daten, Heap und Stack liegen außerhalb der reservierten SRAM4-Bank.

| Aktion | Payload danach | Zustand danach | Privater M7-Antwortwert |
|---|---|---|---|
| M7 initialisiert | 0 | REQUEST_M4 | – |
| M4 antwortet | 1 | RESPONSE_M7 | – |
| M7 liest und veröffentlicht | 2 | REQUEST_M4 | 1 |
| M4 antwortet | 3 | RESPONSE_M7 | – |
| M7 liest und veröffentlicht | 4 | REQUEST_M4 | 3 |
| M4 antwortet | 5 | RESPONSE_M7 | – |
| M7 beendet | 5 | TURN_M7 | 5 |

Bei `REQUEST_M4` darf der M4 die Payload verändern, bei `RESPONSE_M7` der M7. Der vorige Besitzer schreibt nach Übergabe nicht weiter. Veröffentlichung: Payload schreiben, DMB, Zustand schreiben. Konsum: erwarteten Zustand prüfen, DMB, Payload lesen. DMB ist Data Memory Barrier, Datenspeicherbarriere. Genau ein Teilnehmer je Kern wird vorausgesetzt.

`NOLOAD` stellt keine normalen Initialdaten für diese Sektion bereit, verbietet aber nicht beliebige Startup- oder Debugger-Schreibzugriffe. Beide Copy-/Zero-Tabellen müssen `.shared` aussparen. Der M7 richtet den Block einmal ein, während der M4 im vereinbarten Stop-Bootzustand wartet, und gibt ihn erst danach frei. Einseitiger Reset während einer Runde gehört nicht zum Basisvertrag. Eine alte gültige Kennung ist keine neue Sitzung; Wiederanlauf erfolgt hier nach Diagnose mit gemeinsamem Neustart.

## Aufgabe 3: Sichtbarkeit und ganze Cachezeilen · 8 Punkte

Die folgenden Werte sind synthetische Zustände, keine Boardmessungen. Der M7-Daten-Cache besitzt 32-Byte-Zeilen, der M4 kann ihn nicht direkt lesen.

1. M7 schreibt eins nur in eine Write-Back-Kopie, SRAM enthält weiterhin null. Welchen Wert kann M4 lesen? Umgekehrt schreibt M4 eins in SRAM, während M7 eine alte saubere gültige Kopie mit null hat. Was kann M7 lesen? Behebt `volatile` plus DMB beide Fälle? Begründen Sie. (4 P)
2. Ein Puffer beginnt bei `0x38000014` und hat 16 Byte Länge. Bestimmen Sie abgerundeten Wartungsbeginn, aufgerundetes exklusives Ende, Bytezahl und Zahl der berührten Zeilen. (2 P)
3. In einer Zeile liegen die Wörter A und B. Start beider null. M7 ändert in seiner Kopie A auf eins; die Zeile ist dirty, B dort noch null. M4 schreibt neun in B des SRAM. Was kann ein anschließendes Clean der alten M7-Zeile mit B tun? Welche Granularität muss die Besitzregel bei Cachewartung berücksichtigen? (2 P)

**Musterlösung Aufgabe 3**

M4 kann den alten SRAM-Wert null lesen. M7 kann seine alte Cachekopie mit null lesen, obwohl SRAM bereits eins enthält. `volatile` veranlasst den vorgesehenen Compilerzugriff, ein Load kann aber weiterhin aus dem Cache kommen. DMB ordnet relevante Zugriffe und schreibt Dirty-Zeilen nicht automatisch zurück. Clean schreibt geänderte Zeilen zurück, Invalidate verwirft Kopien. Invalidate eigener Dirty-Daten könnte unveröffentlichte Änderungen verlieren.

Die Startadresse liegt zwanzig Byte nach `0x38000000`. Sechzehn Byte Nutzlänge enden exklusiv bei `0x38000024`, also bereits in der zweiten Zeile. Wartungsbeginn: `0x38000000`, exklusives Ende: `0x38000040`, Spanne 64 Byte, zwei Zeilen. Nachbarobjekte dieser Spanne sind mitzudenken.

Die M7-Kopie enthält für A/B eins/null, SRAM nach dem M4-Schreiben null/neun. Das Clean der Dirty-Zeile kann SRAM auf eins/null überschreiben und damit die neuere neun verlieren. Zwei getrennte Wörter sind keine getrennten Cachezeilen. Wartung braucht exklusive Zuständigkeit für die vollständigen betroffenen Zeilen und einen sicheren Veröffentlichungs-/Konsumvertrag, auch für die Kontrollinformationen.

## Aufgabe 4: MPU, Regionspriorität und Fehlerbudget · 8 Punkte

1. Region vier beschreibt SRAM4 ab `0x38000000` mit 64 KiB als Normal, TEX eins, C null, B null, S eins, Lesen/Schreiben und Execute Never. Region sechs überlappt die erste Hälfte als cachebares Normal-Write-Back mit S null. Beide Regionen und alle betreffenden Subregionen sind aktiviert. Welche Region gilt für `ipc` am Anfang? Erzeugt S eins einen M7/M4-Cacheabgleich? Schützt die M7-MPU damit automatisch vor M4- oder DMA-Zugriffen? (3 P)
2. Bestimmen Sie die letzte eingeschlossene Adresse der 64-KiB-Region und den Armv7-M-SIZE-Code mit Größe $= 2^{\mathrm{SIZE}+1}$ Byte. Welche Reihenfolge gilt zwischen MPU-Konfiguration, D-Cache-Aktivierung und Kommunikation? (2 P)
3. M4 wartet im Modul bis zu 2000 Kernel-Ticks, M7 prüft 2000 HAL-Millisekunden. Bei bestätigten 1000 Kernel-Ticks/s: Welche nominellen Budgets sind das? Warum beweisen sie keine zwei Sekunden vollständige IPC-Deadline? Darf der Fehlerpfad den Shared-Block einfach nullen? (3 P)

**Musterlösung Aufgabe 4**

Die passende höher nummerierte Region sechs gewinnt. Die beabsichtigte nicht cachebare Region vier allein ist deshalb kein Nachweis der effektiven Attribute. S bedeutet shareable und aktiviert auf diesem System keinen automatischen M7/M4-Cacheabgleich. Die M7-MPU ist kernlokal und kontrolliert nicht automatisch den anderen Kern oder DMA, Direct Memory Access beziehungsweise direkten Hardware-Speicherzugriff.

$64 \cdot 1024 = 65536 = 2^{16}$ Byte. Die letzte eingeschlossene Adresse ist `0x3800FFFF`, SIZE ist fünfzehn. Am gemeinsamen Neustart zuerst die Region vor D-Cache-Aktivierung konfigurieren. Im kontrollierten Versuch wird der Block bei D-Cache aus initialisiert, dann D-Cache eingeschaltet und der M4 freigegeben. Die laufende Nutzung wird nicht durch spontane Attributumschaltung verändert.

Nominell 2000/1000 = 2 Sekunden M4-Tickbudget und 2000 ms gleich zwei Sekunden M7-Budget. Die tatsächliche Prüfung setzt laufende Zeitbasen und erneute Ausführung des Wartecodes voraus. M4-Start, Scheduling, andere Unterbrechungen und die definierte Gesamtreaktion sind damit nicht automatisch vollständig begrenzt; Tickzeit ist kein unabhängiger Watchdog. Bei Timeout wird lokal diagnostiziert und angehalten, nicht ohne Besitz `memset` ausgeführt. Die Gegenstelle könnte noch arbeiten. Der erneute Basisversuch benutzt gemeinsamen Reset.

## Teil B: kontrollierter Ping-Pong auf dem NUCLEO-H755ZI-Q

### Aufgabe 5: vorbereitete Projektbasis prüfen

Die Lehrperson stellt eine lauffähige Dual-Core-Basis aus Vorlesung 3–5 mit geprüftem Start bereit. Im heutigen Versuch verwendet CM7 Bare Metal und CM4 genau einen FreeRTOS-Kernel. Vorherige Lastthreads, Button-/UART-Versuche und zusätzliche Schreiber werden deaktiviert. M7 und M4 dürfen dieselbe gemeinsame Takt- oder Peripheriekonfiguration nicht unkoordiniert ändern.

1. Prüfen Sie die .ioc-Konfiguration in STM32CubeMX für Board und STM32H755ZIT6. Erfassen Sie tatsächliche Kerntakte, Startadressen, Bootoptionen und Softwareversionen. Spezifikationsmaxima sind keine Laborfrequenzen.
2. Prüfen Sie die beiden CMake-Ziele in VS Code mit STM32 VS Code Extension. Startups, Vektortabellen und Linkerdateien gehören jeweils zum passenden Ziel. Bestätigen Sie die verwendeten Kernkennungen `CORE_CM7` beziehungsweise `CORE_CM4`.
3. Bestätigen Sie den vorhandenen Boot-HSEM-Pfad: M4 bereitet Benachrichtigung vor und geht in Stop; M7 wartet, initialisiert und gibt frei. Stop ist nicht Reset. HSEM-ID, Taktfreigabe, Take-Status und Wartebedingungen werden aus dem konkreten erzeugten Code geprüft. Frühe Pollbudgets sind nicht ohne Zeitbasis als Millisekunden bezeichnet.
4. Halten Sie die SRAM4-Domäne und benötigten Zugriffswege aktiv. Prüfen Sie die laufende HAL-Millisekundenbasis auf CM7 sowie den M4-Kerneltick. Für den nominalen Budgetvergleich wird ein 32-Bit-Kerneltick mit 1000 Hz gewählt. Der M4-Worker erhält zunächst `osPriorityNormal` und 1024 Byte Stack. Heap, Stack und Threadhandle werden am Projekt geprüft.
5. Prüfen Sie den abgestimmten Flash-/Debugstart für beide Images. Ein Halt beider Kerne schon während des Boot-Wartens kann den erforderlichen Stop- oder Wake-Fortschritt verhindern. Starten Sie beide mit dem erprobten Vorgehen der installierten Erweiterung, ohne aus Erinnerung feste GDB-Ports oder einen zweiten konkurrierenden Debugserver einzusetzen.

### Aufgabe 6: gemeinsames Objekt und MPU integrieren

Die Anwendungsdateien enthalten die vollständige Protokoll- und Fehlerlogik. Für die kurze Kursphase sind der freie MPU-Platz und die Linker-/Startup-Anpassung bereits an der Projektbasis vorbereitet. Studierende prüfen die Ergebnisse gegen Teil A und integrieren die zugeordneten Quellen.

| Datei in `code/lab06/` | Ziel und Rolle |
|---|---|
| `lab06_ipc.h` | beide Ziele, gemeinsamer Vertrag |
| `lab06_shared.c` | genau einmal je Image, Definition von `ipc` |
| `lab06_cm7.c` | ausschließlich CM7, MPU/Init und drei Anfragen |
| `lab06_cm4.c` | ausschließlich CM4, ein CMSIS-Worker |
| `shared-section.ld` | Fragment für beide tatsächlichen Linkerdateien |

1. Fügen Sie `.c`-Dateien und Include-Verzeichnis den passenden CMake-Zielen hinzu. Ein kopierter Dateiname allein nimmt die Quelle nicht in den Build auf. Fügen Sie keinen weiteren Definitionsblock für `ipc` hinzu. Die eigenen statischen und Diagnosevariablen gehören in private Regionen außerhalb SRAM4.

2. Reservieren Sie SRAM4 in beiden tatsächlichen Linkerdateien **einmal** für den Kanal. Im gelieferten Fragment heißt die Region `RAM_SHARED`: Ursprung `0x38000000`, Länge 64 KiB. Ersetzen Sie eine bestehende Zuordnung dieses Bereichs, statt einen überlappenden Alias einzufügen. Keine `.bss`, `.data`, Heap oder Stacks in dieser Region. Das Fragment ist keine komplette Linkerdatei und wird in den vorhandenen `SECTIONS`-Block integriert.
3. Prüfen Sie beide erzeugten Maps und ELF-Dateien: `ipc` an `0x38000000`, Typ zwölf Byte, gepolsterte `.shared` 32 Byte und `NOLOAD`. Die Assertions des Fragments müssen bestehen. Prüfen Sie außerdem Copy-/Zero-Tabellen beider Startups auf Ausschluss der Region. `NOLOAD` verbietet keinen beliebigen späteren Schreibzugriff.
4. Wählen Sie eine **tatsächlich freie** MPU-Regionsnummer, deren Priorität oberhalb widersprechender überlappender Regionen liegt. Lassen Sie die vorhandene grundlegende MPU-Politik der Projektbasis erhalten. Der Helfer setzt für den Kanal Lesen/Schreiben und Execute Never und benutzt `MPU_PRIVILEGED_DEFAULT`; das muss zur privilegierten M7-Bare-Metal-Basis passen. Er erkennt nicht automatisch jede Überlappung.
5. Rufen Sie auf CM7 nach dessen vorhandener MPU-Konfiguration und **vor D-Cache-Aktivierung** auf:

```
Lab06_CM7_ConfigureMemory(shared_region_number, LAB06_NONCACHED);
```

`shared_region_number` ist Ihre zuvor bestätigte freie Nummer. Ersetzen Sie die Angabe durch den tatsächlich gewählten Wert oder eine entsprechende Projektkonstante. Es ist kein universell vorgeschriebener Regionsplatz. Die Referenz prüft numerische Gültigkeit und ausgeschalteten D-Cache. Die effektiven Registerattribute werden zusätzlich kontrolliert.

6. Viele Bootvorlagen haben einen frühen Cachehelper, der I- und D-Cache einschaltet. Für diesen Versuch wird **nur die D-Cache-Aktivierung einmal nach die Shared-Initialisierung gelegt**. Prüfen Sie auch `SystemInit` und weitere MPU-/Cachehelper. Ein nachträglich nochmals aufgerufener MPUhelper darf die Shared-Attribute nicht überschreiben. Die Konfigurationsänderung erfolgt im vorbereiteten Cold-Start-Ablauf, nicht live mit laufender Kommunikation.
7. Fügen Sie nach der gemeinsamen Taktinitialisierung, bei weiterhin ausgeschaltetem D-Cache und vor dem **vorhandenen Boot-HSEM-Release** ein:

```
Lab06_CM7_InitShared();
SCB_EnableDCache();
/* Danach der vorhandene geprüfte HSEM-Bootpfad. */
```

Die Bootfreigabe wird nicht ein zweites Mal implementiert. Prüfen Sie die konkrete Codeposition: Ein allgemeiner USER-CODE-Block nach der Peripherieanlage kann bereits hinter dem HSEM-Release liegen und damit zu spät sein. Die Initialdaten werden so selbst für den späteren Fehlerbuild zuerst im SRAM bereitgestellt. 8. Rufen Sie `Lab06_CM7_Run()` nach der vorhandenen M4-Freigabe und dem erforderlichen lokalen Start **einmal** auf, nicht bei jedem Umlauf der Hauptschleife. Die Funktion kehrt nach drei Antworten zurück. Der normale Boot- und Fehlerpfad der Basis bleibt erhalten. 9. Auf CM4 ruft die einzige erzeugte IPC-Worker-Einstiegsfunktion `Lab06_CM4_Run(argument)` auf. Übernehmen Sie den tatsächlichen Argumentnamen. Keine zusätzliche gleichnamige Eintrittsfunktion und kein zweites `osKernelInitialize` oder `osKernelStart`. Nach der dritten Antwort beendet die Funktion ihren Thread mit `osThreadExit`. CM4 initialisiert oder nullt `ipc` nicht. 10. Bauen und flashen Sie die beiden passenden Images. Prüfen Sie auch nach erneuter CubeMX-Generierung, ob Quellen, Includes, Linkerwahl und Benutzerbereiche erhalten sind. Ein Buildfehler oder ein nicht durchgeführter Lauf wird entsprechend dokumentiert.

**Musterlösung Aufgaben 5–6**

Die vollständigen Anwendungsfunktionen stehen in `code/lab06/`. Der Datenvertrag bewahrt das aus Vorlesung 6 bekannte Layout: magic/state/payload mit Offsets 0/4/8. Die Kennung ist `IPC_MAGIC`, die Phase ein ausgerichtetes 32-Bit-Wort. Diese Zugriffskonvention ist eine ausdrückliche Armv7-M-/Toolchain-Vereinbarung, keine portable ISO-C-Thread-Synchronisation.

`Lab06_CM7_ConfigureMemory` konfiguriert eine passende Normal-Region mit TEX eins, C null, B null und S eins und erhält die anderen Regionsbeschreibungen. Seine Enable-Politik muss zur Basis passen. Vor jeder Verwendung werden Priorität und effektive Attribute geprüft. `Lab06_CM7_InitShared` verlangt konfigurierte Region und D-Cache aus. M4 ist noch im Boot-Stop; die Funktion setzt Kennung, Payload null und ersten Request geordnet. Erst danach wird D-Cache eingeschaltet und der vorhandene HSEM-Pfad freigegeben.

Auf CM4 startet der Kernel den einzigen Worker. Der Worker prüft zuerst Kennung und Phase, dann nach DMB den erwarteten geraden Anfragewert. Er antwortet mit dem nächsten ungeraden Wert und gibt den Zustand an M7. M7 kopiert die bestätigte Antwort privat und stellt nur die ersten beiden Folgeanfragen bereit. Nach der dritten Antwort bleibt Payload fünf, Zustand TURN_M7. Die private Diagnose wird nicht als zusätzlicher unkoordinierter Shared-Schreiber verwendet.

Zwei getrennte Images dürfen je eine Definition desselben physischen Objekts enthalten. Zwei Definitionen innerhalb eines Images wären dagegen ein Integrationsfehler. Anwendungsquellen allein erzeugen weder Flashstart noch Taktbasis, Linkerskript oder Bootvektoren. Diese projektspezifischen Bestandteile werden am tatsächlich verwendeten Projekt nachgewiesen.

### Aufgabe 7: Basislauf und absichtlichen Fehlerbuild vergleichen

**Basislauf, gemeinsamer Reset:**

- M7-Memorymode `LAB06_NONCACHED`; andere M7-Regionen dürfen D-Cache nutzen.
- Nach Abschluss müssen CM7-Diagnose `lab06_cm7_reply` mit 1, 3, 5 und `lab06_cm7_done` mit eins sowie CM4-Diagnose `lab06_cm4_responses` mit drei und `lab06_cm4_done` mit eins zusammenpassen.
- `lab06_error` ist ein privates Symbol in jedem Image, nicht ein gemeinsamer Fehlerzähler. Beide lokalen Werte sind im gültigen Lauf `LAB06_ERROR_NONE`.
- Gemeinsamer Endblock: gültige Kennung, Payload fünf, Zustand `TURN_M7`. Der M4-Worker ist beendet, der M7 hat keinen vierten Request veröffentlicht.
- Halten Sie für eine konsistente Momentaufnahme alle Kerne an, nachdem beide Abschlusskennzeichen gesetzt sind. Halten verändert das ursprüngliche Timing. Eine angehaltene Gegenstelle mitten im Warten ist kein unabhängiger Cachebeweis.

**Fehlerbuild, eigener gemeinsamer Reset:**

Ändern Sie ausschließlich den M7-Modus bei `Lab06_CM7_ConfigureMemory` auf `LAB06_CACHED_FAULT`. Der Helfer wählt TEX eins, C eins, B eins, S null, also cachebares Normal-Write-Back. Initialisierung bleibt bei D-Cache aus, danach D-Cache tatsächlich einschalten. M4-Code, Layout und Phasenfolge bleiben gleich. Es gibt bewusst keine Cachewartung an diesem gemischten Block.

1. Prüfen Sie effektive MPU-Attribute, aktive Region, Subregionen und Cachebit im M7-CCR, Configuration and Control Register. Fehlende Aktivierung oder eine höhere Überlappung wäre ein anderer Versuch.

2. Dokumentieren Sie die tatsächlich erreichten Antworten, Phase und lokale Fehlercodes. Möglich sind veraltete Werte oder Timeout; ein vollständiger Stillstand oder identischer Fehler in jedem Build wird nicht vorausgesetzt. Bei erfolgreichem Lauf prüfen Sie ebenfalls die Voraussetzungen, bevor Sie einen sicheren Write-Back-Kanal behaupten.
3. Erklären Sie, welche zwei Sichtbarkeitsfehler zum beobachteten Verlauf passen könnten. Eine SRAM-Debugansicht ist nicht automatisch die Kopie, aus der ein laufender M7-Load gelesen hat.
4. Bei Fehler **keine spontane Neuinitialisierung**, keine laufende MPU-Umschaltung und kein pauschales Clean/Invalidate des Blocks. Die andere Seite könnte noch Besitzer sein oder Daten geschrieben haben. Diagnostizieren Sie und starten Sie für eine neue Runde gemeinsam.
5. Stellen Sie `LAB06_NONCACHED` wieder her, bauen Sie neu und prüfen Sie den Basislauf erneut.

| Kontrolle | Eigener Nachweis / Beobachtung |
|---|---|
| tatsächliches Board, Takte, Versionen | |
| Boot-HSEM-ID und Startreihenfolge | |
| beide Maps und Startup-Ausschluss | |
| M7-Region, Priorität und Cachebit | |
| CM4-Tickfrequenz, Threadhandle und Stack | |
| Basis: Antworten und Abschluss beider Kerne | |
| Fehlerbuild: Verlauf und lokale Fehlercodes | |
| wiederhergestellte Basis | |

**Musterlösung Aufgabe 7**

Der gültige Basislauf erfüllt die logische Folge $0 \to 1 \to 2 \to 3 \to 4 \to 5$ mit den drei privaten Antworten 1, 3 und 5. Nach der letzten Antwort bleibt fünf stehen, kein Payload sechs und kein weiterer Request. Genau ein Berechtigter schreibt je Phase. `volatile` und DMB bleiben nötig, obwohl der kleine Kanal nicht cachebar ist.

Im Fehlerbuild kann M7 noch eine alte Zustandskopie lesen, während M4 bereits geantwortet hat. Oder eine Folgeanfrage bleibt nur im Dirty-Cache, während M4 alten SRAM sieht. Cacheverdrängung und konkrete Startreihenfolge können den Fehler verdecken. Ein einmaliger Erfolg widerlegt das Sichtbarkeitsproblem nicht.

Die lokale Timeoutprüfung verlangt fortschreitende Zeit und erneute Codeausführung. `HAL_Delay(1)` ist keine M4-Kernelblockierung; `osDelay(1)` zählt einen Kernel-Tick. Weder deren Parameter noch das Diagnosefenster liefern eine vollständige IPC-Latenz. Der tatsächliche Boardlauf bleibt ohne Projekt/Hardware als nicht durchgeführt gekennzeichnet.

### Zusatzaufgabe: eine echte Zeitmessung definieren

Nur mit passendem Messgerät oder Trace. Legen Sie ein Anforderungsereignis und den geforderten Abschluss fest. Wählen Sie nach Board-/Pinprüfung getrennte GPIO-Marker für den Anfang und das Ende und eine gemeinsame Zeitbasis. Schalten Sie die Marker an den tatsächlichen Veröffentlichungs-/Antwortstellen; berücksichtigen Sie ihre Kosten. Nehmen Sie UART-Ausgabe und absichtliche Anzeige-/Pollingpausen nicht unbemerkt als IPC-Eigenschaft in die Interpretation auf.

Dokumentieren Sie Auflösung, Samplezahl, Last, Kerntakte, Tickrate, Optimierung, Cache-/MPU-Attribute und Debugzustand. Subtrahieren Sie nicht einfach zwei lokale Zykluszähler

ohne bestätigte Skalierung und Synchronisation. Der erste Durchlauf kann Boot- und Kernelstart enthalten; spätere Runden sind ein anderer Messfall. Ein endlicher beobachteter Höchstwert ist noch kein allgemeiner Worst-Case-Nachweis.

Ohne Messung bleiben Mikrosekundenwerte leer. Eine sinnvoll definierte Messskizze und die begründete Daten-/Fehlerauswertung sind dann die Abgabe.

## Abgabe und Quellen

Teil A: begründete Lösungen zu Aufgaben 1–4, 32 Punkte. Teil B: Konfiguration, Quellenintegration, Linker-/Startup- und Regenerierungsprüfung, Basis-/Fehlerbuildvergleich sowie wiederhergestellte Basis. Trennen Sie Modellrechnung, erwartet, tatsächlich beobachtet und nicht durchgeführt.

Quellen: Vorlesung 6; Giorgio C. Buttazzo, *Hard Real-Time Computing Systems*, 3. Auflage; ST RM0399, UM2408, AN5617, AN4838 und AN5361; eingebundene STM32H7-HAL- und CMSIS-M7-Header sowie die Dokumentation der installierten STM32 VS Code Extension. Das öffentlich eingesehene H745-Boottemplate illustriert das Muster; die H755-Projektbasis und ihre Versionen bestimmen die tatsächliche Integration.